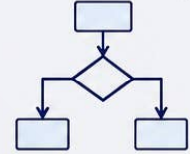


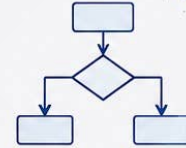
1010101010
0101010101
1010101010
0101010101



Notes



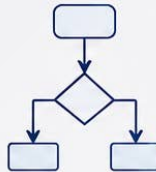
1010101010
0101010101
1010101010
0101010101



Unit-I



10110
01001
10101



Exception Handling & Multithreaded Programming

1. Exception Handling

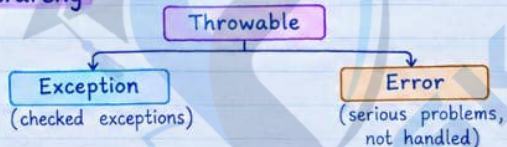
1. Meaning

- Exception handling is a mechanism to handle runtime errors or unexpected situations in a program.
- It allows the program to continue normal flow instead of terminating abruptly.

2. Types of Exceptions

- Checked Exception** → Must be handled or declared. (e.g. IOException, SQLException)
- Unchecked Exception** → Runtime exception, not mandatory to handle. (e.g. NullPointerException, ArithmeticException)

3. Hierarchy



4. Keywords

- try** → contains risky code
- catch** → handles the exception
- finally** → always executes (optional)
- throw** → explicitly throws an exception
- throws** → declares the exception

6. Benefits

- Maintains normal flow of program.
- Provides error information.
- Improves code reliability and robustness.

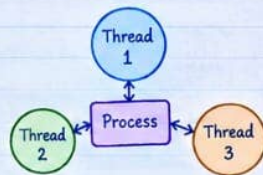
5. Example

```
public class Example {
    public static void main(String[] args) {
        try {
            int data = 10 / 0;
        } catch (ArithmeticException e) {
            System.out.println("Cannot divide by zero");
        } finally {
            System.out.println("Finally block executed");
        }
    }
}
```

Multithreaded Programming

1. Meaning

- A thread is a lightweight process.
- It allows a program to do multiple tasks at the same time.
- In Java, a thread is a separate path of execution.



2. Ways to Create a Thread

(i) By extending Thread class

```
class MyThread extends Thread {
    public void run() {
        System.out.println("Thread is running");
    }
}

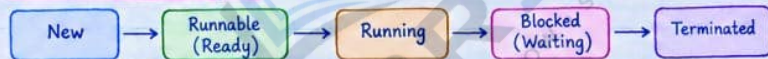
public class Main {
    public static void main(String[] args) {
        MyThread t = new MyThread();
        t.start(); // starts a new thread
    }
}
```

(ii) By implementing Runnable interface

```
class MyRunnable implements Runnable {
    public void run() {
        System.out.println("Thread is running");
    }
}

public class Main {
    public static void main(String[] args) {
        Thread t = new Thread(new MyRunnable());
        t.start();
    }
}
```

3. Thread Life Cycle



4. Important Methods

- start()** → starts the thread
- run()** → contains the code to be executed
- sleep(long millis)** → pauses the thread
- join()** → waits for another thread to finish
- isAlive()** → checks if thread is alive

5. Synchronization

- Used to control access to shared resources.
- Prevents data inconsistency.
- Methods:
 - synchronized block
 - synchronized method

```
public class Main {
    public static void main(String[] args) {
        Thread t1 = new Thread(new Counter());
        Thread t2 = new Thread(new Counter());
        t1.start();
        t2.start();
    }
}
```

Conclusion :

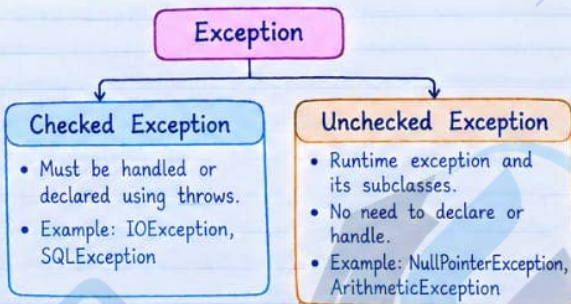
Exception handling helps to manage runtime errors and maintain a smooth flow of the program. Multithreading increases the efficiency of the application by allowing multiple tasks to run simultaneously.

Exception Handling : Concept of Exception

1. What is Exception?

- An exception is an event that occurs during the execution of a program which disrupts the normal flow of instructions.
- It is an unwanted or unexpected condition that prevents the program from running normally.
- In Java, exceptions are represented as objects of classes that extend the `Throwable` class.

2. Types of Exceptions



3. Characteristics

- It interrupts the normal flow of the program.
- It is an object.
- It contains information about the error.
- It can be caught and handled using `try-catch` block.
- It can be propagated using `throws` keyword.

4. Example

```
public class Example {  
    public static void main(String[] args) {  
        try {  
            int data = 10 / 0; // ArithmeticException  
            System.out.println("Result: " + data);  
        } catch (ArithmeticException e) {  
            System.out.println("Cannot divide by zero");  
        }  
        finally {  
            System.out.println("Program ended.");  
        }  
    }  
}
```

5. How Exception Works?

- 1 When an error occurs, the JVM creates an exception object.
- 2 The exception is thrown using `throw` (automatically by JVM in some cases).
- 3 The control is transferred to the nearest `catch` block.
- 4 If no `catch` block is found, the program terminates with an error.

6. Keywords

try

Contains the code that may throw an exception.

catch

Handles the exception.

finally

Always executes (optional).

throw

Explicitly throws an exception.

throws

Declares the exception.

7. Benefits

- Maintains normal flow of program.
- Provides error information.
- Improves code reliability and robustness.

Conclusion :

Exception is an important concept in Java that helps to handle runtime errors gracefully and keeps the program running smoothly.



Handle
Errors,
Build Better
Programs

Types of Exception, Try, Catch and Finally

1. Types of Exception

In Java, exceptions are mainly divided into two types:

Exception

Checked Exception

- Must be handled or declared using throws.
- They are checked at compile time.
- Examples:
 - IOException
 - SQLException
 - ClassNotFoundException

Unchecked Exception

- Runtime exceptions and errors (not checked at compile time).
- They are optional to handle.
- Examples:
 - NullPointerException
 - ArithmeticException
 - ArrayIndexOutOfBoundsException

2. Example of Types of Exception

Checked Exception Example

```
import java.io.*;
class CheckedExExample {
    public static void main(String[] args) {
        try {
            FileInputStream file = new FileInputStream("test.txt");
        } catch (IOException e) {
            System.out.println("File not found");
        }
    }
}
```

→ IOException is a checked exception (must be handled or declared).

Unchecked Exception Example

```
class UncheckedExExample {
    public static void main(String[] args) {
        int a = 10, b = 0;
        try {
            int c = a / b;
        } catch (ArithmeticException e) {
            System.out.println("Cannot divide by zero");
        }
    }
}
```

→ ArithmeticException is an unchecked exception (runtime exception).

3. Try, Catch and Finally

Try

- The try block contains the code that may throw an exception.
- If an exception occurs, the control moves to the corresponding catch block.

```
try {
    int data = 10 / 0; // This will throw ArithmeticException
    System.out.println("Result: " + data);
}
```

Put the code that may cause an exception inside try block.

Catch

- The catch block catches and handles the exception.
- We can have multiple catch blocks for different types of exceptions.

```
catch (ArithmeticException e) {
    System.out.println("Cannot divide by zero");
}
```

Specify the exception type that you want to handle.

Finally

- The finally block is always executed (whether an exception occurs or not).
- It is usually used to release resources (e.g., closing a file, database connection).

```
finally {
    System.out.println("Execution completed.");
}
```

Use finally for cleanup code.

Summary

- Try → contains code that may throw an exception.
- Catch → handles the exception.
- Finally → always executes, used for cleanup.

- Checked exceptions → must be handled or declared.
- Unchecked exceptions → optional to handle (runtime).

Multiple Catch Blocks, Nested Try Statements, throw & throws

1. Multiple Catch Blocks

Concept

- Java allows multiple catch blocks to handle different types of exceptions.
- The catch blocks are checked in the order they are written.
- The first matching catch block is executed.
- A general exception class (Exception) should be placed at the end (optional).

Example

```
public class MultipleCatchExample {
    public static void main(String[] args) {
        try {
            int a = 10 / 0; // ArithmeticException
        } catch (ArithmeticException e) {
            System.out.println("Arithmetic error: " + e);
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Array index error: " + e);
        } catch (Exception e) {
            System.out.println("General error: " + e);
        }
    }
}
```



Note:

- More specific exceptions should be caught before more general ones.
- The order of catch blocks matters.

2. Nested Try Statements

Concept

- A try block can be placed inside another try block.
- It is used when one operation depends on another operation.
- The inner try-catch handles the specific exception, and the outer catch handles other exceptions.

Example

```
public class NestedTryExample {
    public static void main(String[] args) {
        try {
            int[] arr = new int[5];
            try {
                arr[5] = 10; // ArrayIndexOutOfBoundsException
            } catch (ArrayIndexOutOfBoundsException e) {
                System.out.println("Inner catch: " + e);
            }
        } catch (Exception e) {
            System.out.println("Outer catch: " + e);
        }
    }
}
```



Key Point:

The inner try-catch handles exceptions related to the inner block, while the outer catch handles exceptions from the outer block.

3. throw and throws

A. throw (Keyword)

- Used to explicitly throw an exception.
- It is used inside a method body.
- It creates an exception object and passes it to the calling code.

```
public class ThrowExample {
    public static void checkAge(int age) {
        if (age < 18) {
            throw new ArithmeticException("Age must be 18+");
        }
    }
}
```

B. throws (Keyword)

- Used in method signature to declare that a method may throw an exception.
- It does not throw the exception, just informs the caller.
- The actual exception is thrown using the throw keyword inside the method.

```
public class ThrowsExample {
    public static void readFile() throws IOException {
        FileReader fr = new FileReader("data.txt");
        // may throw IOException
    }
}
```



Remember:

- throw → throws an exception.
- throws → declares an exception.

Conclusion :



Multiple catch blocks help handle different exceptions, nested try statements manage dependent operations, while throw and throws provide a way to explicitly raise and declare exceptions in Java.

Multithreading

1. Thread Basics

1. What is a Thread?

- A thread is a lightweight process.
- It allows a program to do multiple tasks at the same time.
- In Java, a thread is a separate path of execution.

2. Why Use Threads?

- To utilize CPU efficiently.
- To improve performance and responsiveness.
- To perform multiple tasks simultaneously (concurrent execution).

3. Creating a Thread

There are two ways to create a thread in Java:

(i) By extending Thread class

```
class MyThread extends Thread {  
    public void run() {  
        System.out.println("Thread is running");  
    }  
    public static void main(String[] args) {  
        MyThread t = new MyThread();  
        t.start(); // starts a new thread  
    }  
}
```

(ii) By implementing Runnable interface

```
class MyRunnable implements Runnable {  
    public void run() {  
        System.out.println("Thread is running");  
    }  
    public static void main(String[] args) {  
        Thread t = new Thread(new MyRunnable());  
        t.start(); // starts a new thread  
    }  
}
```

4. Key Methods

- **start()** → starts the thread and calls the run() method.
- **run()** → contains the code to be executed by the thread.
- **sleep(long millis)** → pauses the thread for the specified time.
- **join()** → waits for another thread to finish.
- **isAlive()** → checks if the thread is alive.



Note:

The run() method is not called directly. It is called automatically by the JVM when start() is invoked.

2. Thread Life Cycle

1. What is Thread Life Cycle?

- It is the different states a thread goes through from its creation to its termination.
- A thread changes its state based on the events and operations it performs.

2. Thread States



3. Summary

- **New** → Thread is created but not yet started.
- **Runnable** → Ready to run (waiting for CPU).
- **Running** → Thread is executing.
- **Blocked** → Waiting for a resource (e.g., I/O, lock).
- **Waiting** → Waiting indefinitely (e.g., wait()).
- **Terminated** → Execution is completed or stopped.



Remember:

A thread moves between states based on events like **start()**, **sleep()**, **wait()**, **notify()**, **join()** and resource availability.

Concept of Multithreading & Creating & Running Threads

1. Concept of Multithreading

What is Multithreading?

- Multithreading is a technique in which a single program is able to execute multiple threads (smallest unit of execution) concurrently (at the same time).
- Each thread runs independently and shares the same memory space and resources with other threads.
- It helps in better utilization of CPU and improves the performance and responsiveness of the application.

Key Points

- A thread is a lightweight process.
- Multiple threads can run within a single JVM process.
- Threads share the same heap memory but have their own stack.
- Multithreading is useful for tasks like background processing, UI responsiveness, downloading, etc.

Example

In a web browser, one thread may handle user input, another thread may load a page in the background.

Advantage

- Better CPU utilization
- Improved performance
- More responsive applications
- Concurrent execution of tasks

Disadvantage

- Complexity in program design
- Synchronization issues
- Deadlocks (possible)
- Harder to debug and test

Conclusion :

→ Multithreading allows multiple threads to run concurrently in a single program, making better use of system resources and improving performance, but it also introduces complexity and synchronization challenges.

2. Creating & Running Threads

1. Methods to Create a Thread

1) By extending Thread class

- Create a class that extends Thread and override the `run()` method.

```
class MyThread extends Thread {  
    public void run() {  
        System.out.println("Thread is running");  
    }  
}
```

2) By implementing Runnable interface

- Create a class that implements Runnable and override the `run()` method.

```
class MyRunnable implements Runnable {  
    public void run() {  
        System.out.println("Thread is running");  
    }  
}
```

2. Running the Thread

- Create an object of the thread class.
- Call the `start()` method (not `run()`).
- The JVM will then call the `run()` method automatically.

Example 1 : Using Thread class

```
public class ThreadExample {  
    public static void main(String[] args) {  
        MyThread t = new MyThread();  
        t.start(); // starts a new thread  
    }  
}
```

Example 2 : Using Runnable interface

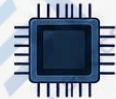
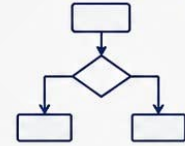
```
public class RunnableExample {  
    public static void main(String[] args) {  
        MyRunnable r = new MyRunnable();  
        Thread t = new Thread(r);  
        t.start(); // starts a new thread  
    }  
}
```



Note :

- Always use `start()` to begin a thread, not `run()`.
- The `run()` method contains the code to be executed by the thread.
- A thread can be started only once.

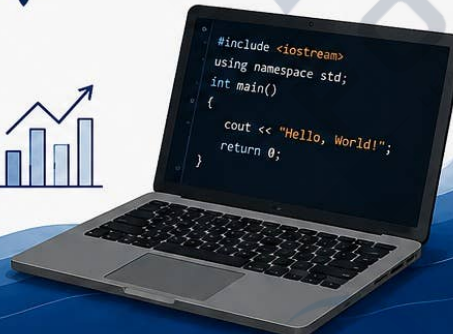
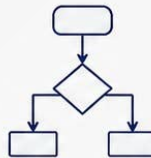
1010101010
0101010101
1010101010
0101010101



Unit - II



10110
01001
10101



Applet Programming : Introduction to Applet

1. What is an Applet?

- An applet is a small Java program that runs inside a web browser.
- It is designed to provide interactive and dynamic content on a web page.
- Applets are part of the Java Foundation Classes (JFC).
- They are executed in a secure environment (sandbox) to protect the user's system.

4. How Applet Works?

- 1 The applet class is written in Java and extends the Applet class (or JApplet class).
- 2 The applet is compiled to a .class file.
- 3 The .class file is embedded in an HTML page using <applet> tag.
- 4 When the web page is loaded, the browser's JVM (with a plugin) runs the applet.

5. Life Cycle of an Applet

An applet has a life cycle with 4 main methods:

- 1 `init()` - called once, for initialization.
- 2 `start()` - called when the applet becomes visible.
- 3 `paint()` - used for drawing on the screen.
- 4 `stop()` - called when the applet is no longer visible.
- 5 `destroy()` - called before the applet is destroyed.

2. Key Features

- Runs inside a web browser.
- Platform independent (Write Once, Run Anywhere).
- Provides interactive graphics and animation.
- Uses the AWT (Abstract Window Toolkit) or Swing for GUI components.
- Secure (restricted access to system resources).
- Small in size and easy to embed in HTML pages.

6. Example : Simple Applet

```
import java.applet.Applet;
import java.awt.Graphics;

public class MyApplet extends Applet {
    public void init() {
        System.out.println("Applet Initialized");
    }
    public void start() {
        System.out.println("Applet Started");
    }
    public void paint(Graphics g) {
        g.drawString("Hello, Applet!", 50, 50);
    }
    public void stop() {
        System.out.println("Applet Stopped");
    }
}
```

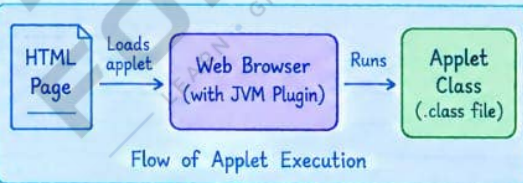
This code displays "Hello, Applet!" on the browser window.

3. Applet vs Application

Feature	Applet	Application
Execution	In web browser	Standalone JVM
Entry point	<code>init()</code> , <code>start()</code>	<code>main()</code>
GUI	Uses AWT/Swing	Uses AWT/Swing
Security	Sandbox (restricted)	Full access
Usage	Web pages	Local machine

7. HTML Page to Run Applet

```
<html>
<head>
<title>Applet Example</title>
</head>
<body>
<applet code="MyApplet.class" width="300" height="100">
</applet>
</body>
</html>
```



Conclusion :

An applet is a simple Java program that runs in a web browser and provides interactive features. It is platform independent, secure and follows a well-defined life cycle.



Applet Life Cycle

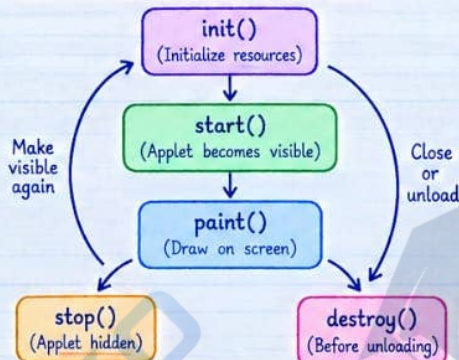
1. What is Applet Life Cycle ?

- The applet life cycle refers to the sequence of steps that an applet goes through from its creation to its destruction.
- These steps are handled by specific methods in the Applet class.
- The life cycle is managed by the Java Applet viewer or a web browser.

2. Life Cycle Stages

- 1 **init()** - Called once, when the applet is initialized.
- 2 **start()** - Called when the applet becomes visible (or starts).
- 3 **paint()** - Called to draw the applet on the screen.
- 4 **stop()** - Called when the applet is no longer visible (or stopped).
- 5 **destroy()** - Called before the applet is destroyed (just before unloading).

3. Applet Life Cycle Diagram



4. Example

```
import java.applet.Applet;
import java.awt.Graphics;
public class LifeCycleApplet extends Applet {
    public void init() {
        System.out.println("init() called");
    }
    public void start() {
        System.out.println("start() called");
    }
    public void paint(Graphics g) {
        System.out.println("paint() called");
    }
    public void stop() {
        System.out.println("stop() called");
    }
    public void destroy() {
        System.out.println("destroy() called");
    }
}
```

5. Methods and Their Functions

Method	When it is called	Purpose
init()	Once, at creation	Initialize the applet and resources.
start()	When applet becomes visible	Start the applet (e.g., start threads, load data).
paint()	Whenever applet needs to be drawn	Draw the applet on the screen.
stop()	When applet is hidden	Pause the applet (e.g., stop threads).
destroy()	Before applet is removed	Release resources and clean up.

6. Key Points

- **init()** is called only once.
- **start()** and **stop()** are called multiple times.
- **paint()** can be called many times (e.g., when the screen is refreshed).
- **destroy()** is called only once, at the end.
- The applet life cycle is controlled by the browser or applet viewer, not by the applet itself.



Remember :

init() → start() → paint() → stop() → destroy()
(This is the general flow, not always strict.)

Conclusion :

The applet life cycle defines the sequence of methods that run from initialization to destruction, helping the applet manage its resources and display content properly.



Difference Between Application & Applet

1. Introduction

- Both Application and Applet are Java programs, but they differ in their purpose, execution environment and structure.
- An application runs as a standalone program, while an applet runs inside a web browser.

3. Key Differences

Feature	Application	Applet
Definition	Complete <u>Java</u> program with main() method.	Small program that runs inside a web browser.
Execution Environment	Runs on JVM independently.	Runs inside a web browser (applet container).
Entry Point	main() method	init() method (and paint()).
Access to System Resources	Can access system resources (with proper permissions).	Restricted access (security sandbox).
Dependency	Not dependent on a browser.	Dependent on a web browser and Java <u>plugin</u> .
Purpose	Standalone desktop or console application.	Interactive content on a web page (e.g., animation, graphics).
User Interface	Uses AWT/Swing (or other GUI).	Uses AWT/Swing, but runs in browser window.
File Structure	.java file → compiled to .class → run with JVM.	.java file → compiled to .class → embedded in HTML and run by browser.

2. What is an Application?

- An application is a complete Java programs with a main() method.
- It runs independently on the JVM (Java Virtual Machine).
- It is not dependent on a web browser.
- It can access system resources directly (with proper permissions).

Example (Application)

```
public class HelloApp {  
    public static void main(String[] args) {  
        System.out.println("Hello, Application!");  
    }  
}
```

4. Example (Applet)

```
<html>  
  <head>  
    <title>Applet Example</title>  
  </head>  
  <body>  
    <applet code="MyApplet.class" width="300" height="150">  
    </applet>  
  </body>  
</html>
```

5. Summary

- Application:** Runs independently, has main(), full system access.
- Applet:** Runs in a browser, has init(), paint(), restricted access.

Conclusion

The main difference is that an application runs as a standalone program, while an applet runs inside a web browser with restricted access and no main() method.



HTML Applet Tag with All Attributes

1. What is <applet> Tag?

- The <applet> tag is used in HTML to embed a Java applet in a web page.
- It acts as a container for the applet, and its attributes provide information about the applet, such as the class file, size, and other settings.

2. Syntax

```
<applet attribute1="value1"  
attribute2="value2"  
...  
></applet>
```

3. Example

```
<applet code="MyApplet.class"  
width="300" height="150"  
name="MyApplet"  
align="center"  
archive="applet.jar"  
MAYSCRIPT>
```

Your browser does not support Java applets.
</applet>

4. All Attributes of <applet> Tag

Attribute	Description	Example
code	Specifies the Java class file (applet class) to load.	code="MyApplet.class"
codebase	Specifies the URL of the directory where the class file is located (optional).	codebase="classes/"
archive	Specifies the list of JAR files containing the applet (optional, multiple files separated by commas).	archive="applet.jar, lib.jar"
width	Specifies the width of the applet display area (in pixels).	width="300"
height	Specifies the height of the applet display area (in pixels).	height="150"
name	Assigns a name to the applet (can be used in JavaScript).	name="MyApplet"
align	Specifies the alignment of the applet in the page (left, right, top, middle, bottom).	align="center"
hspace	Specifies the horizontal space (in pixels) around the applet.	hspace="10"
vspace	Specifies the vertical space (in pixels) around the applet.	vspace="10"
border	Specifies the border width (in pixels) around the applet.	border="1"
codeType	Specifies the MIME type of the applet class file (e.g., application/x-java-applet).	codeType="application/x-java-applet"
MAYSCRIPT	Allows the applet to execute JavaScript (deprecated).	MAYSCRIPT
ALT	Provides alternate text if the applet cannot be displayed.	ALT="Java applet"



Note:

- Multiple JAR files can be specified in the archive attribute.
- The codebase attribute is useful when the applet class file is not in the same directory as the HTML page.
- The MAYSCRIPT attribute is rarely used and is considered outdated.



Summary

The <applet> tag and its attributes help to embed and configure a Java applet in a web page. Each attribute provides specific information about the applet's class, size, location, and behavior.

5. Important Points

- <applet> tag is obsolete in modern HTML5.
- It was widely used in older web pages to run Java applets.
- Modern web applications use technologies like JavaScript, HTML5 canvas, and WebGL instead of applets.

Different Applet Methods

1. What are Applet Methods?

- Applet methods are special functions provided by the `java.applet.Applet` class (or its subclasses).
- These methods are called automatically at different stages of the applet life cycle or can be invoked by the browser.
- They help in initializing, starting, stopping and destroying the applet, and in handling events and user interaction.

3. List of Important Applet Methods

Category	Method	Description	Example
Life Cycle Methods	<code>init()</code>	Called once when the applet is initialized.	<pre>public void init() { // initialization code }</pre>
	<code>start()</code>	Called when the applet becomes visible (or starts).	<pre>public void start() { // start code }</pre>
	<code>stop()</code>	Called when the applet is no longer visible (or stopped).	<pre>public void stop() { // stop code }</pre>
	<code>destroy()</code>	Called before the applet is destroyed (just before unloading).	<pre>public void destroy() { // cleanup code }</pre>
UI / Display Methods	<code>paint(Graphics g)</code>	Used for drawing the applet's output.	<pre>public void paint(Graphics g) { // drawing code }</pre>
	<code>update(Graphics g)</code>	Called when the applet needs to be refreshed.	<pre>public void update(Graphics g) { // refresh code }</pre>
	<code>getParameter(String name)</code>	Returns the value of a parameter from HTML.	<code>String value = getParameter("name");</code>
Event Handling Methods	<code>mouseDown(Event evt, int x, int y)</code>	Called when a mouse button is pressed.	<pre>public boolean mouseDown(Event evt, int x, int y) { ... }</pre>
	<code>mouseUp(Event evt, int x, int y)</code>	Called when a mouse button is released.	<pre>public boolean mouseUp(Event evt, int x, int y) { ... }</pre>
	<code>keyDown(Event evt, int key)</code>	Called when a key is pressed.	<pre>public boolean keyDown(Event evt, int key) { ... }</pre>
Utility Methods	<code>getAppletInfo()</code>	Returns information about the applet.	<pre>public String getAppletInfo() { return "My Applet"; }</pre>
	<code>getCodeBase()</code>	Returns the base URL of the applet.	<pre>public URL getCodeBase() { ... }</pre>
	<code>getDocumentBase()</code>	Returns the URL of the HTML page.	<pre>public URL getDocumentBase() { ... }</pre>

2. Major Categories of Applet Methods

- ① Life cycle methods (lifecycle control)
- ② UI / Display methods (drawing and appearance)
- ③ Event handling methods (mouse, keyboard, etc.)
- ④ Utility methods (information and settings)

4. Example Usage

```
import java.applet.Applet;  
import java.awt.*;  
  
public class MyApplet extends Applet {  
    public void init() {  
        setBackground(Color.LIGHT_BLUE);  
    }  
    public void paint(Graphics g) {  
        g.drawString("Hello, Applet!", 50, 50);  
    }  
}
```

5. Key Points

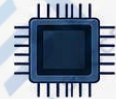
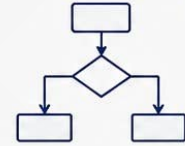
- `init()` is used for initialization.
- `start()` and `stop()` control visibility.
- `destroy()` is used for cleanup.
- `paint()` is used for drawing.
- Event methods handle user interaction.
- `getParameter()` is used to read values from HTML.



Note:

Different applet methods are called automatically by the browser at appropriate times, and they work together to provide an interactive web experience.

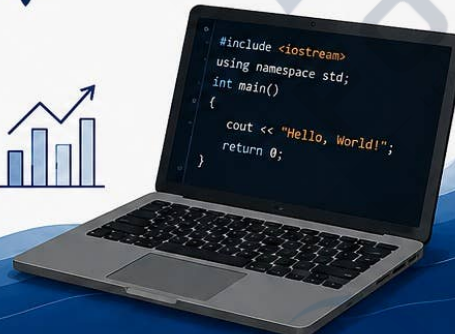
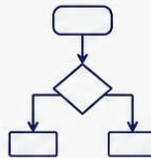
1010101010
0101010101
1010101010
0101010101



Unit - II



10110
01001
10101



Event Handling : Event Delegation Model

1. What is Event Delegation Model?

- The Event Delegation Model is the mechanism used in AWT and Swing for handling events in Java.
- It separates the event source (where the event occurs) and the event listener (who handles the event).
- The event is generated by the source and delegated to the listener through an event object.
- The listener registers with the source to receive events.

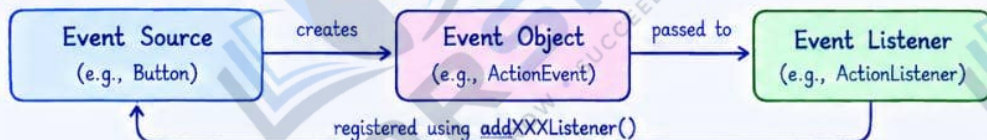
2. Components of Event Delegation Model

- ① **Event Source**
 - The component where the event occurs (e.g., Button, TextField, etc.).
- ② **Event Object**
 - Contains information about the event (e.g., source, type, time).
- ③ **Event Listener**
 - The object that handles the event. (e.g., ActionListener, MouseListener, etc.).
- ④ **Registering**
 - The listener is registered with the source using addXXXListener() method.

3. Working of Event Delegation Model

- ① An event occurs on the source component.
- ② The source creates an event object (e.g., ActionEvent, MouseEvent).
- ③ The event is passed to the registered listener (via the event handling method).
- ④ The listener object contains the code to handle the event.

5. Diagram of Event Delegation Model



6. Key Points to Remember

- The event delegation model is based on the observer pattern.
- It provides a loose coupling between the source and listener.
- The listener must implement the appropriate listener interface.
- The addXXXListener() method is used to register the listener.
- Common listener interfaces: ActionListener, MouseListener, KeyListener, ItemListener, etc.



Note:

- Event handling in Java is simple, flexible and powerful due to the event delegation model.
- It allows the same listener to handle events from multiple sources.

4. Example : ActionEvent (Button Click)

```
import java.awt.*;
import java.awt.event.*;

class EventDemo implements ActionListener {
    public void actionPerformed(ActionEvent e) {
        System.out.println("Button Clicked!");
    }
}

public class Main {
    public static void main(String[] args) {
        Frame f = new Frame("Event Delegation");
        Button b = new Button("Click Me");
        b.addActionListener(new EventDemo()); // register f.add(b);
        f.setSize(300, 200);
        f.setLayout(null);
        b.setBounds(100, 100, 100, 50);
        f.setVisible(true);
    }
}
```

Event Classes

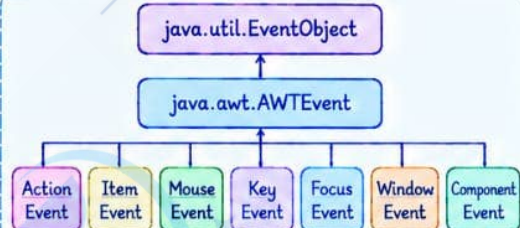
1. What are Event Classes?

- Event classes represent the events that occur in a GUI application.
- Each event class is a subclass of `java.util.EventObject`.
- They contain information about the event, such as the source of the event, time, and modifiers (e.g., mouse button, key pressed, etc.).

2. Categories of Event Classes

- ① Action Events (e.g., `ActionEvent`)
- ② Item Events (e.g., `ItemEvent`)
- ③ Mouse Events (e.g., `MouseEvent`)
- ④ Key Events (e.g., `KeyEvent`)
- ⑤ Focus Events (e.g., `FocusEvent`)
- ⑥ Window Events (e.g., `WindowEvent`)
- ⑦ Component Events (e.g., `ComponentEvent`)

3. Hierarchy of Event Classes



4. Common Event Classes and Their Details

Event Class	Package	Description	Common Methods (examples)
<code>ActionEvent</code>	<code>java.awt.event</code>	Generated when an action occurs (e.g., button click).	<code>getActionCommand()</code> , <code>getSource()</code>
<code>ItemEvent</code>	<code>java.awt.event</code>	Generated when the state of an item changes (e.g., checkbox, list).	<code>getStateChange()</code> , <code>getItem()</code>
<code>MouseEvent</code>	<code>java.awt.event</code>	Generated when mouse is used (e.g., click, move, press, release).	<code>getX()</code> , <code>getY()</code> , <code>getButton()</code> , <code>getClickCount()</code>
<code>KeyEvent</code>	<code>java.awt.event</code>	Generated when a key is pressed, released or typed.	<code>getKeyCode()</code> , <code>getKeyChar()</code> , <code>getModifiers()</code>
<code>FocusEvent</code>	<code>java.awt.event</code>	Generated when a component gains or loses focus.	<code>getID()</code> , <code>getOppositeComponent()</code>
<code>WindowEvent</code>	<code>java.awt.event</code>	Generated when a window is opened, closed, activated, etc.	<code>getWindow()</code> , <code>getID()</code>
<code>ComponentEvent</code>	<code>java.awt.event</code>	Generated when a component is shown, hidden, moved or resized.	<code>getComponent()</code> , <code>getID()</code>

5. Example : `ActionEvent` Class

```
import java.awt.*;
import java.awt.event.*;

public class MyButton extends Frame implements ActionListener {
    Button b;
    public MyButton() {
        b = new Button("Click Me");
        b.addActionListener(this);
        add(b);
        setSize(300, 200);
        setLayout(null);
        setVisible(true);
    }

    public void actionPerformed(ActionEvent e) {
        System.out.println("Button clicked!");
    }
}
```

6. Key Points

- Event classes are subclasses of `java.util.EventObject`.
- Each event class carries event-specific information.
- They are used with event listener interfaces to handle events.



Note:

- Event classes are part of the AWT and Swing libraries.
- For example, `ActionEvent` is used with `ActionListener`, `MouseEvent` with `MouseListener`, etc.

Event Listener Interfaces

1. What are Event Listener Interfaces?

- Event listener interfaces are special interfaces provided by the Java AWT and Swing packages.
- They contain abstract methods that are invoked when a specific event occurs.
- Classes that are interested in receiving events implement these interfaces and register with the event source.

4. Details of Important Event Listener Interfaces

Listener Interface	Package	Main Methods	Example Event(s)
ActionListener	java.awt.event	void actionPerformed(ActionEvent e)	Button click, Menu selection
ItemListener	java.awt.event	void itemStateChanged(ItemEvent e)	Checkbox, Choice, List
MouseListener	java.awt.event	void mouseClicked(MouseEvent e) void mousePressed(MouseEvent e) void mouseReleased(MouseEvent e) void mouseEntered(MouseEvent e) void mouseExited(MouseEvent e)	Mouse click, press, release, enter, exit
KeyListener	java.awt.event	void keyPressed(KeyEvent e) void keyReleased(KeyEvent e) void keyTyped(KeyEvent e)	Key press, key release, key type
FocusListener	java.awt.event	void focusGained(FocusEvent e) void focusLost(FocusEvent e)	Focus gained, focus lost
WindowListener	java.awt.event	void windowOpened(WindowEvent e) void windowClosing(WindowEvent e) void windowClosed(WindowEvent e) void windowIconified(WindowEvent e) void windowDeiconified(WindowEvent e) void windowActivated(WindowEvent e) void windowDeactivated(WindowEvent e)	Window open, close, minimize, maximize, activate, deactivate

2. Common Event Listener Interfaces

- 1** ActionListener – for action events (e.g., button click)
- 2** ItemListener – for item state events (e.g., checkbox, choice)
- 3** MouseListener – for mouse events (e.g., click, press, release)
- 4** KeyListener – for keyboard events (e.g., key press, key release)
- 5** FocusListener – for focus events (e.g., gained, lost)
- 6** WindowListener – for window events (e.g., open, close, resize)

3. Example (ActionListener)

```
import java.awt.event.*;
import javax.swing.*;

public class MyFrame extends JFrame
    implements ActionListener {
    JButton btn;

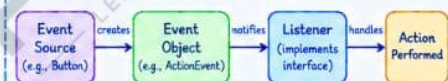
    public MyFrame() {
        btn = new JButton("Click Me");
        btn.addActionListener(this);
        add(btn);
        setSize(300, 200);
        setLayout(null);
        setVisible(true);
    }

    public void actionPerformed(ActionEvent e) {
        System.out.println("Button clicked!");
    }
}
```

5. Event Flow

- Event occurs in the GUI component (source).
- The event object is created (e.g., ActionEvent).
- The registered listener's method is called.
- The application performs the required action.

6. Simple Diagram



Note

- Listener interfaces are part of the java.awt.event and javax.swing.event packages.
- We usually implement only the methods we need (adapter classes can be used for convenience).
- Multiple listeners can be registered on the same component.

Event Listener Interfaces
make your GUI responsive!

Handling Mouse and Keyboard Events

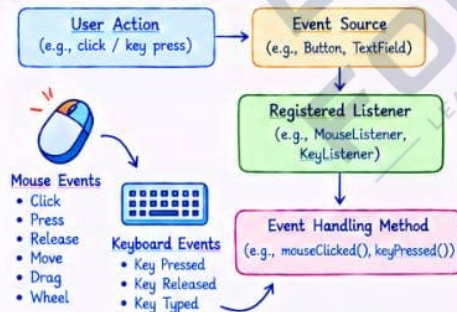
1. Introduction

- Mouse and keyboard events are used to detect and respond to user actions like clicking, moving the mouse, pressing keys, etc.
- These events are handled using event listener interfaces provided by Java AWT and Swing.
- The event source (e.g., Button, TextField, Panel) registers the listener using `addXxxListener()` method.
- When the event occurs, the corresponding method of the listener interface is called.

4. Registering the Listener

- The listener object is registered using the `addMouseListener()`, `addMouseMotionListener()`, `addMouseWheelListener()` or `addKeyListener()` method.
- Example:
`button.addMouseListener(new MyMouseListener());`
`textField.addKeyListener(new MyKeyListener());`

5. Diagram : Event Flow



2. Mouse Event Handling

Mouse events are handled using the `MouseListener`, `MouseMotionListener` and `MouseWheelListener` interfaces.

MouseListener Methods

Method	Description
<code>mouseClicked(MouseEvent e)</code>	Called when a mouse button is clicked.
<code>mousePressed(MouseEvent e)</code>	Called when a mouse button is pressed.
<code>mouseReleased(MouseEvent e)</code>	Called when a mouse button is released.
<code>mouseEntered(MouseEvent e)</code>	Called when the mouse enters the component.
<code>mouseExited(MouseEvent e)</code>	Called when the mouse exits the component.

MouseMotionListener Methods

Method	Description
<code>mouseMoved(MouseEvent e)</code>	Called when the mouse is moved.
<code>mouseDragged(MouseEvent e)</code>	Called when the mouse is dragged.

MouseWheelListener Method

Method	Description
<code>mouseWheelMoved(MouseWheelEvent e)</code>	Called when the mouse wheel is rotated.

6. Complete Example : Handling Mouse & Keyboard Events

```
import java.awt.*;
import java.awt.event.*;
public class InputDemo extends Frame implements
    MouseListener, KeyListener {
    TextField text;
    public InputDemo() {
        text = new TextField(20);
        add(text);
        addMouseListener(this);
        addKeyListener(this);
        setSize(400, 200);
        setLayout(null);
        setVisible(true);
    }
    // Mouse events
    public void mouseClicked(MouseEvent e) {
        text.setText("Mouse Clicked!");
    }
    // Key events
    public void keyPressed(KeyEvent e) {
        text.setText("Key Pressed: " + e.getKeyChar());
    }
    public void keyReleased(KeyEvent e) {
        text.setText("Key Released: " + e.getKeyChar());
    }
    public void keyTyped(KeyEvent e) {
        text.setText("Key Typed: " + e.getKeyChar());
    }
    public static void main(String[] args) {
        new InputDemo();
    }
}
```

3. Keyboard Event Handling

Keyboard events are handled using the `KeyListener` interface.

KeyListener Methods

Method	Description
<code>keyPressed(KeyEvent e)</code>	Called when a key is pressed.
<code>keyReleased(KeyEvent e)</code>	Called when a key is released.
<code>keyTyped(KeyEvent e)</code>	Called when a character is typed (used for text input).

Example : Handling Mouse Events (AWT)

```
import java.awt.*;
import java.awt.event.*;
public class MouseDemo extends Frame implements MouseListener {
    TextField text;
    public MouseDemo() {
        text = new TextField(20);
        add(text);
        addMouseListener(this);
        setSize(300, 200);
        setLayout(null);
        setVisible(true);
    }
    public void mouseClicked(MouseEvent e) {
        text.setText("Mouse Clicked!");
    }
    public static void main(String[] args) {
        new MouseDemo();
    }
}
```

7. Important Points

- `MouseListener` handles mouse click, press, release, enter and exit events.
- `MouseMotionListener` handles mouse move and drag events.
- `MouseWheelListener` handles mouse wheel rotation.
- `KeyListener` handles key press, release and type events.
- Only one listener object can be registered for each component (use adapter classes if needed).
- Events are handled on the Event Dispatch Thread (EDT) in Swing (use `SwingUtilities.invokeLater()` if required).



Note

For real applications, it is recommended to use Adapter classes (e.g., `MouseAdapter`, `KeyAdapter`) to avoid implementing all methods of the interfaces.

Adapter Classes

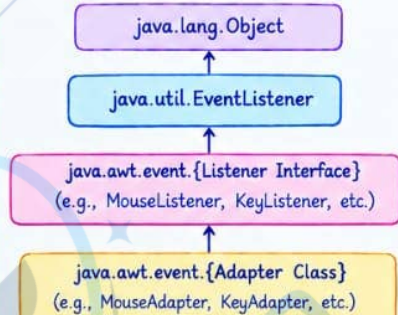
1. What are Adapter Classes?

- Adapter classes are abstract classes provided by the Java AWT and Swing packages.
- They implement the respective listener interfaces with empty method bodies (default implementations).
- They allow you to override only the methods you need, which makes the code simpler and easier to maintain.
- Adapter classes are used when an event listener interface has multiple methods, but you are interested in only one or a few of them.

2. Types of Adapter Classes

- 1 **WindowAdapter**
- Implements WindowListener (for window events)
- 2 **MouseAdapter**
- Implements MouseListener + MouseMotionListener (for mouse events)
- 3 **KeyAdapter**
- Implements KeyListener (for keyboard events)
- 4 **ComponentAdapter**
- Implements ComponentListener (for component events)

3. Class Hierarchy



4. Common Adapter Classes and Their Purpose

Adapter Class	Implements	Purpose
WindowAdapter	WindowListener	Handles window events (open, close, minimize, maximize, etc.).
MouseAdapter	MouseListener + MouseMotionListener	Handles mouse events (click, press, release, move, drag, etc.).
KeyAdapter	KeyListener	Handles keyboard events (key pressed, released, typed, etc.).
ComponentAdapter	ComponentListener	Handles component events (resize, move, show, hide, etc.).

5. Example : Using MouseAdapter

```
import java.awt.*;
import java.awt.event.*;

public class MouseAdapterDemo extends Frame {
    public MouseAdapterDemo() {
        setTitle("MouseAdapter Example");
        setSize(400, 300);
        setLayout(null);
        addMouseListener(new MyMouseAdapter());
        setVisible(true);
    }

    public static void main(String[] args) {
        new MouseAdapterDemo();
    }
}

class MyMouseAdapter extends MouseAdapter {
    public void mouseClicked(MouseEvent e) {
        System.out.println("Mouse clicked at: " +
            e.getX() + ", " + e.getY());
    }
}
```

6. Key Points

- Adapter classes are abstract classes, not interfaces.
- They provide empty implementations of all methods in the listener interface.
- You can extend the adapter class and override only the required methods.
- They make the code shorter and more readable.
- Widely used in AWT and Swing event handling.

7. Advantages

- Reduces the number of methods to implement.
- Simplifies event handling code.
- Improves code readability and maintainability.
- Useful when you are interested in only specific events.



Note

Adapter classes are particularly useful when working with complex event listener interfaces like MouseListener, KeyListener, and WindowListener.

AWT (Abstract Window Toolkit)

— Java's original GUI toolkit for creating graphical user interfaces —

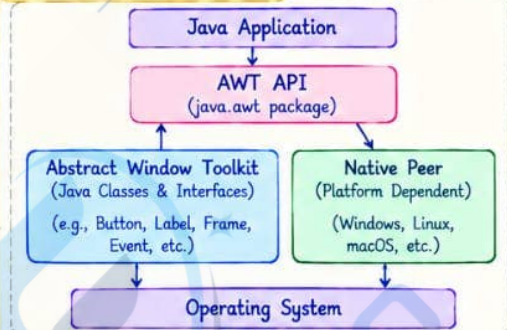
1. What is AWT?

- AWT (Abstract Window Toolkit) is a set of classes and interfaces provided by Java to create graphical user interfaces (GUI) and to handle user events.
- It is part of the `java.awt` package.
- AWT provides a platform-independent API for building GUI applications.
- It uses native system resources (peer components) to create actual GUI components.

2. Key Features of AWT

- ✓ Provides a rich set of graphical components (buttons, labels, text fields, etc.).
- ✓ Event-driven programming model.
- ✓ Platform independent (uses native peers).
- ✓ Simple and easy to use.
- ✓ Forms the basis for Swing (Swing is built on AWT).
- ✓ Supports both heavyweight and lightweight components.

3. AWT Architecture



4. Important AWT Components

Component Class	Description
Frame	Main window with title bar and border.
Button	Push button.
Label	Displays a short text.
TextField	Single line text input field.
TextArea	Multi line text input field.
Checkbox	Allows multiple selections.
Choice	Drop-down list.
List	Displays a list of items.
Scrollbar	Provides a scroll bar.
Panel	A container for other components.

5. AWT Event Handling

- AWT uses an event-driven model.
- Events are generated by user actions (e.g., click, key press, mouse movement).
- Listeners are used to handle these events.
- Common event classes: `ActionEvent`, `KeyEvent`, `MouseEvent`, etc.
- Common listener interfaces: `ActionListener`, `KeyListener`, `MouseListener`, etc.



Note:

- AWT is an older API. In modern Java development, Swing is more commonly used because it provides a richer set of components and better look and feel.

6. Example : Creating a Frame with a Button

```
import java.awt.*;
import java.awt.event.*;

public class AwtExample extends Frame implements ActionListener {
    Button btn;

    public AwtExample() {
        btn = new Button("Click Me");
        btn.setBounds(50, 50, 100, 30);
        btn.addActionListener(this);
        add(btn);
        setTitle("AWT Example");
        setSize(300, 200);
        setLayout(null);
        setVisible(true);
    }

    public void actionPerformed(ActionEvent e) {
        btn.setLabel("Button Clicked!");
    }

    public static void main(String[] args) {
        new AwtExample();
    }
}
```

7. Summary

- AWT is a fundamental GUI toolkit in Java, providing basic components, event handling, and a platform-independent interface for building GUI applications.

AWT Class Hierarchy

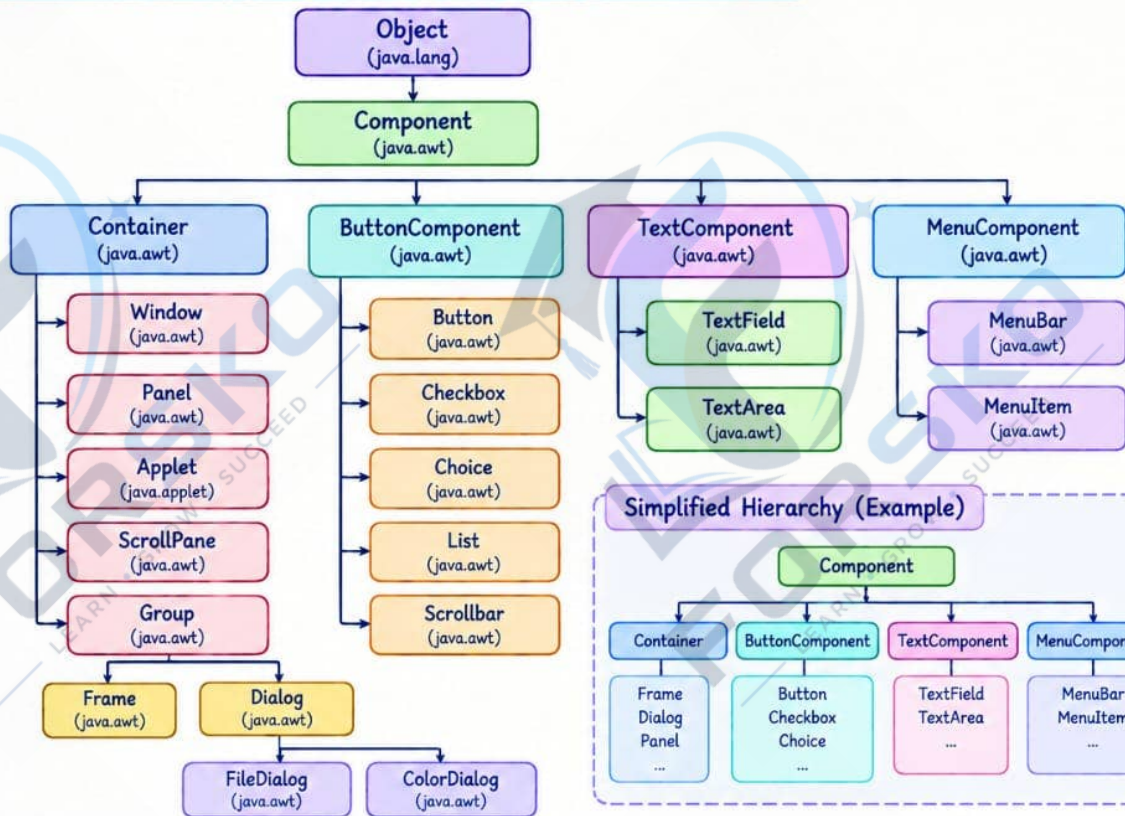
1. What is AWT?

- AWT is a part of the Java standard library (java.awt package).
- It provides classes and interfaces to create GUI components like buttons, labels, text fields, etc.
- It uses native system resources (peer components) to create actual GUI elements.

2. Key Points

- The top-most class in AWT is `java.awt.Component`.
- All GUI components are subclasses of `Component`.
- `Container` is a subclass of `Component` and can hold other components.
- `Window` is a subclass of `Container` and represents a top-level window.
- Some classes (like `Frame`, `Dialog`, etc.) provide windows with title bars.

The AWT (Abstract Window Toolkit) provides a set of classes and interfaces for creating graphical user interfaces (GUI) in Java.



Note: In later versions of Java (Swing), many AWT components are still used, but Swing provides more advanced and flexible components.

Components, Containers, Frames, Panels, Window

1. Components

- Components are the basic building blocks of a GUI (Graphical User Interface) in AWT.
- They are used to create interactive elements like buttons, labels, text fields, checkboxes, etc.
- All visual elements (except containers) are components.

Common Components in AWT:

- Button
- Label
- TextField
- TextArea
- Checkbox
- Choice
- List
- Scrollbar
- Menu
- MenuItem

Example:

```
import java.awt.*;

public class ComponentExample {
    public static void main(String[] args) {
        Button btn = new Button("Click");
        Label lbl = new Label("Name:");
        TextField txt = new TextField(15);
        Checkbox chk = new Checkbox("Agree");

        Frame f = new Frame("Component Demo");
        f.setLayout(new FlowLayout());
        f.add(lbl);
        f.add(txt);
        f.add(btn);
        f.add(chk);
        f.setSize(300,150);
        f.setVisible(true);
    }
}
```

2. Containers

- A Container is a special type of Component that can hold other components or containers inside it.
- Containers provide a space to organize and manage child components.
- Examples of containers in AWT are Frame, Panel, Window, Dialog, Applet, etc.

Container Hierarchy (basic):



Example:

```
Frame f = new Frame("My Frame");
Panel p = new Panel();
f.add(p);
f.setLayout(new FlowLayout());
f.setSize(300,200);
f.setVisible(true);
```

3. Frames

- A Frame is a top-level window with a title bar, border and minimize, maximize, close buttons.
- It is used as the main window of an application.
- Frame is a subclass of Window.

Key Points:

- Provides the main application window.
- Can contain other components or containers.
- Used when you want a standalone window with title and system controls.

Example:

```
Frame f = new Frame("My Frame");
f.setSize(400,300);
f.setLayout(new FlowLayout());
f.setVisible(true);
```



4. Panels

- A Panel is a lightweight container.
- It is used to group components within a frame or another container.
- Panels do not have title bars or system controls.
- It helps in organizing the GUI into sections.

Key Points:

- Used for grouping related components.
- Can be added to a Frame, Window or another Panel.
- Useful for complex layouts.

Example:

```
Frame f = new Frame("Panel Example");
Panel p = new Panel();
p.setLayout(new GridLayout(2,1));
p.add(new Button("B1"));
p.add(new Button("B2"));
f.add(p);
f.setSize(300,200);
f.setVisible(true);
```



5. Window

- A Window is a top-level container without a title bar and border (unlike Frame).
- It is used for creating custom windows or dialog boxes.
- It does not have minimize, maximize or close buttons.
- It is a superclass of Frame.

Key Points:

- Used for custom windows and dialogs.
- Usually has no title bar or border.
- Can contain components or other containers.

Example:

```
Window w = new Window(new Frame());
w.setSize(300,200);
w.setLayout(new FlowLayout());
w.add(new Label("This is a Window"));
w.setVisible(true);
```



AWT Controls : Button, Label, Text Field and Layout Managers

1. Button

- A Button is a clickable component used to perform an action.
- It generates an ActionEvent when clicked.
- Class: java.awt.Button
- Common methods: setLabel(), getLabel(), addActionListener()

Example:

```
import java.awt.*;
import java.awt.event.*;

public class ButtonExample extends Frame {
    Button btn;

    public ButtonExample() {
        setTitle("Button Example");
        setSize(300, 200);
        setLayout(null);
        btn = new Button("Click Me");
        btn.setBounds(100, 80, 100, 30);
        btn.addActionListener(e ->
            setTitle("Button Clicked!"));
        add(btn);
        setVisible(true);
    }

    public static void main(String[] args) {
        new ButtonExample();
    }
}
```



(Button)

2. Label

- A Label is a non-editable component used to display text or information.
- It does not generate any event.
- Class: java.awt.Label
- Common methods: setText(), getText(), setAlignment()

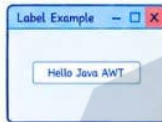
Example:

```
import java.awt.*;

public class LabelExample extends Frame {
    Label lbl;

    public LabelExample() {
        setTitle("Label Example");
        setSize(300, 200);
        setLayout(null);
        lbl = new Label("Hello Java AWT");
        lbl.setBounds(80, 80, 150, 30);
        add(lbl);
        setVisible(true);
    }

    public static void main(String[] args) {
        new LabelExample();
    }
}
```



(Label)

3. Text Field

- A Text Field is a single-line input component used to take user input.
- It generates an ActionEvent when Enter is pressed.
- Class: java.awt.TextField
- Common methods: setText(), getText(), setColumns()

Example:

```
import java.awt.*;
import java.awt.event.*;

public class TextFieldExample extends Frame {
    TextField tf;

    public TextFieldExample() {
        setTitle("Text Field Example");
        setSize(300, 200);
        setLayout(null);
        tf = new TextField(20);
        tf.setBounds(80, 80, 150, 25);
        add(tf);
        setVisible(true);
    }

    public void actionPerformed(ActionEvent e) {
        setTitle("You entered: " + tf.getText());
    }

    public static void main(String[] args) {
        new TextFieldExample();
    }
}
```



(Text Field)

Layout Managers

- Layout managers are used to arrange components inside a container automatically.
- They help in creating platform-independent and flexible GUI layouts.

- Common layout managers in AWT:

- FlowLayout
- BorderLayout
- GridLayout
- CardLayout

1. FlowLayout

- Places components in a row, wrapping to next line if needed.
- Default layout manager.

Example:

```
import java.awt.*;

public class FlowLayoutExample extends Frame {
    public FlowLayoutExample() {
        setTitle("FlowLayout Example");
        setSize(300, 150);
        setLayout(new FlowLayout());
        add(new Button("One"));
        add(new Button("Two"));
        add(new Button("Three"));
        setVisible(true);
    }

    public static void main(String[] args) {
        new FlowLayoutExample();
    }
}
```



(FlowLayout)

2. BorderLayout

- Divides the container into five regions: North, South, East, West, Center.
- Only one component can be placed in each region.

Example:

```
import java.awt.*;

public class BorderLayoutExample extends Frame {
    public BorderLayoutExample() {
        setTitle("BorderLayout Example");
        setSize(400, 200);
        setLayout(new BorderLayout());
        add(new Button("North"), BorderLayout.NORTH);
        add(new Button("South"), BorderLayout.SOUTH);
        add(new Button("East"), BorderLayout.EAST);
        add(new Button("West"), BorderLayout.WEST);
        add(new Button("Center"), BorderLayout.CENTER);
        setVisible(true);
    }

    public static void main(String[] args) {
        new BorderLayoutExample();
    }
}
```



(BorderLayout)

3. GridLayout

- Arranges components in a grid (rows and columns).
- All components are of equal size.

Example:

```
import java.awt.*;

public class GridLayoutExample extends Frame {
    public GridLayoutExample() {
        setTitle("GridLayout Example");
        setSize(300, 200);
        setLayout(new GridLayout(2, 2));
        add(new Button("1"));
        add(new Button("2"));
        add(new Button("3"));
        add(new Button("4"));
        setVisible(true);
    }

    public static void main(String[] args) {
        new GridLayoutExample();
    }
}
```



(GridLayout)

4. CardLayout

- Stacks components like cards.
- Only one component is visible at a time.
- Useful for creating multiple views in the same container.

Example:

```
import java.awt.*;
import java.awt.event.*;

public class CardLayoutExample extends Frame {
    CardLayout card = new CardLayout();
    Panel panel = new Panel();

    public CardLayoutExample() {
        setTitle("CardLayout Example");
        setSize(300, 200);
        setLayout(null);
        panel.setLayout(card);
        panel.add("card1", new Label("First Card"));
        panel.add("card2", new Label("Second Card"));
        add(panel);
        setVisible(true);
    }

    public static void main(String[] args) {
        new CardLayoutExample();
    }
}
```

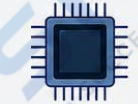
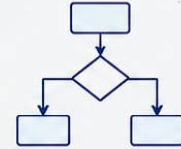


(CardLayout)

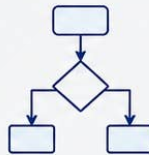


Note: Layout managers make the GUI more flexible and platform-independent. Choose the right layout manager based on your design needs.

1010101010
0101010101
1010101010
0101010101



10110
01001
10101



Unit - IV

JDBC (Java Database Connectivity)

JDBC Concept

JDBC (Java Database Connectivity) is a Java API that provides a standard interface for connecting and interacting with databases from a Java application.

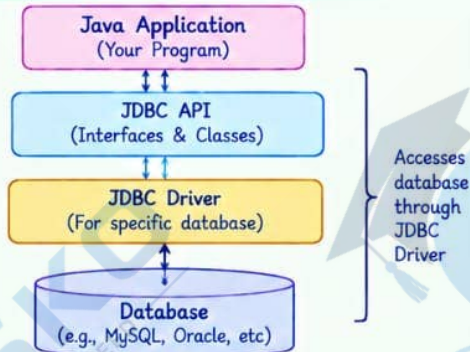
1. What is JDBC?

- JDBC is a specification (API) provided by Sun Microsystems (now Oracle).
- It enables Java programs to connect with relational databases.
- It provides a set of classes and interfaces to execute SQL queries and retrieve data.
- JDBC is database-independent, meaning the same code can work with different databases (with appropriate drivers).

4. Working Steps

- Load the driver (optional in newer versions).
- Establish a connection using DriverManager.
- Create a statement / prepared statement.
- Execute the SQL query.
- Process the result using ResultSet.
- Close the resources (ResultSet, Statement, Connection).

2. Architecture / Working of JDBC



3. Key Components

- DriverManager**
 - Manages the available database drivers and establishes a connection.
- Connection**
 - Represents a connection with the database.
- Statement / PreparedStatement**
 - Used to execute SQL queries.
- ResultSet**
 - Holds the data returned from the database.
- SQLException**
 - Handles database-related errors.

5. Example : JDBC Code

```
import java.sql.*;

public class JdbcExample {
    public static void main(String[] args) {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/testdb", "root", "password");
            Statement stmt = con.createStatement();
            ResultSet rs = stmt.executeQuery("SELECT * FROM student");
            while(rs.next()) {
                System.out.println(rs.getInt("id") + " " + rs.getString("name"));
            }
            rs.close(); stmt.close(); con.close();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

6. JDBC Drivers

- Type 1 - JDBC-ODBC Bridge Driver** (legacy, not commonly used)
- Type 2 - Native API Driver** (uses native libraries)
- Type 3 - Network Protocol Driver** (middle-tier server)
- Type 4 - Thin Driver** (pure Java, most commonly used)

Example: MySQL uses Type 4 driver (com.mysql.cj.jdbc.Driver).

7. Advantages

- Provides a standard interface for multiple databases.
- Keeps the application code independent of the database.
- Easy to use and learn.
- Supports transactions and batch processing.
- Widely used in enterprise applications.

8. Conclusion

JDBC makes it simple for Java applications to connect with and manipulate data from databases using a consistent and standardized API.

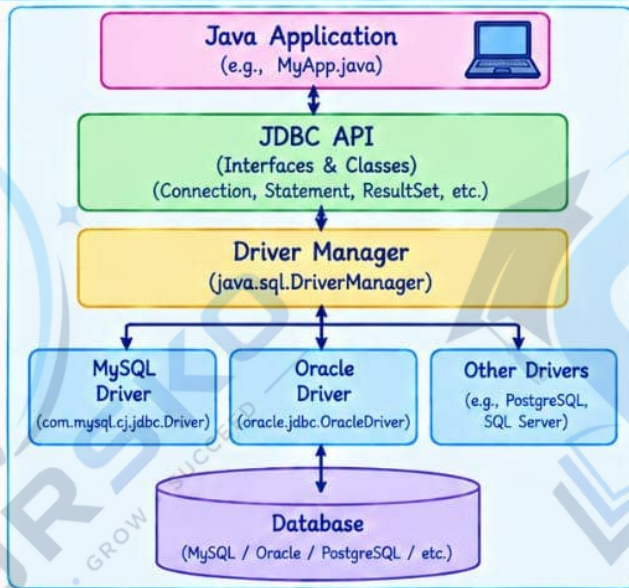
JDBC = Java + Database + Connectivity

JDBC Architecture

JDBC (Java Database Connectivity) architecture defines how Java applications interact with a database using a set of API, driver manager and database drivers.

Key Components

- 1 Java Application**
 - Contains the business logic and uses JDBC API to access the database.
- 2 JDBC API**
 - Provides standard interfaces and classes (e.g., Connection, Statement, ResultSet) to interact with the database.
- 3 Driver Manager**
 - Loads and manages the appropriate database driver based on the URL.
- 4 JDBC Driver**
 - Vendor-specific driver that translates JDBC calls into database-specific protocol.
- 5 Database**
 - The actual database where the data is stored.



How It Works (Flow)

- 1** The Java application calls JDBC API (e.g., DriverManager.getConnection()).
- 2** The DriverManager locates and loads the appropriate JDBC driver based on the URL.
- 3** The driver establishes a connection with the database.
- 4** The application uses the Connection object to create a Statement / PreparedStatement.
- 5** The query is executed and the ResultSet is returned.
- 6** The application processes the result and closes the resources (ResultSet, Statement, Connection).

Advantages

- Provides a standard interface for multiple databases.
- Keeps the application code independent of the database.
- Easy to use and learn.
- Supports transactions and batch processing.
- Widely used in enterprise applications.

Example of JDBC Connection

```
import java.sql.*;

public class JdbcExample {
    public static void main(String[] args) {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/testdb", "root", "password");
            System.out.println("Connected!");
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Popular JDBC Drivers

- 1 MySQL Connector/J** (com.mysql.cj.jdbc.Driver)
- 2 Oracle JDBC Driver** (oracle.jdbc.OracleDriver)
- 3 PostgreSQL Driver** (org.postgresql.Driver)
- 4 SQL Server Driver** (com.microsoft.sqlserver.jdbc.SQLServerDriver)
- 5 SQLite Driver** (org.sqlite.JDBC)

JDBC API

(Java Database Connectivity API)

JDBC API provides a set of interfaces and classes in the `java.sql` and `javax.sql` packages that allow Java programs to connect with databases, execute SQL queries, and process the results.

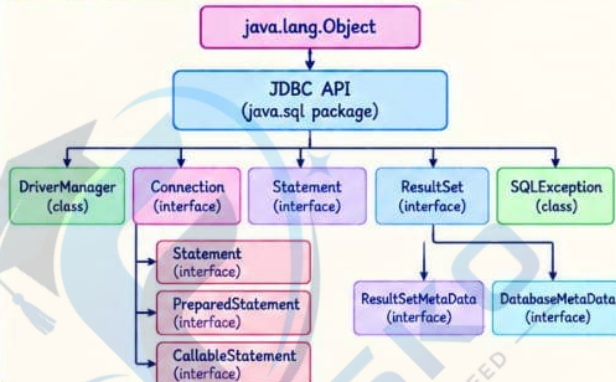
1. What is JDBC API?

- JDBC API is a standard Java API that defines a set of interfaces and classes for database access.
- It is part of the Java Standard Edition (JSE) and mainly present in `java.sql` package (and `javax.sql` for advanced features).
- It allows Java applications to connect to relational databases, execute SQL commands and retrieve results in a database-independent way.

2. Important Interfaces in JDBC API

Interface	Purpose / Description
DriverManager	Manages the list of database drivers and establishes a connection.
Connection	Represents a connection with a database.
Statement	Used to execute simple SQL statements (without parameters).
PreparedStatement	Used to execute precompiled SQL statements (with parameters).
CallableStatement	Used to execute stored procedures.
ResultSet	Holds the data returned from a database query.
ResultSetMetaData	Provides metadata about the ResultSet (columns, data types, etc.).
DatabaseMetaData	Provides metadata about the database (product name, version, tables, etc.).
SQLException	Represents an exception that occurs during database access.

3. JDBC API Hierarchy (Simplified)



4. Steps to Use JDBC API

- Load the JDBC Driver
(Register the driver with DriverManager)
- Establish a Connection
(Use DriverManager.getConnection())
- Create a Statement / PreparedStatement
- Execute SQL Query
(executeQuery() for SELECT, executeUpdate() for INSERT, UPDATE, DELETE)
- Process the Result
(Use ResultSet to retrieve data)
- Close the resources
(ResultSet, Statement, Connection)

5. Example: Using JDBC API

```
import java.sql.*;

public class JdbcExample {
    public static void main(String[] args) {
        try {
            // 1. Load the driver (optional in newer versions)
            Class.forName("com.mysql.cj.jdbc.Driver");
            // 2. Establish connection
            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/testdb", "root", "password");
            // 3. Create statement
            Statement stmt = con.createStatement();
            // 4. Execute query
            ResultSet rs = stmt.executeQuery("SELECT * FROM student");
            // 5. Process result
            while (rs.next()) {
                System.out.println(rs.getInt("id") + " - " + rs.getString("name"));
            }
            // 6. Close resources
            rs.close();
            stmt.close();
            con.close();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

6. Packages Related to JDBC API

- `java.sql` → Contains core JDBC interfaces and classes (DriverManager, Connection, Statement, ResultSet, etc.).
- `javax.sql` → Provides advanced features like connection pooling (DataSource, ConnectionPoolDataSource, etc.).

7. Advantages of JDBC API

- Standard and database-independent.
- Simple and easy to use.
- Supports a wide range of databases.
- Provides flexibility and control over SQL execution.
- Widely used in enterprise applications.

8. Conclusion

JDBC API acts as a bridge between a Java application and a database by providing a set of standard interfaces and classes for database access.

Types of JDBC Drivers

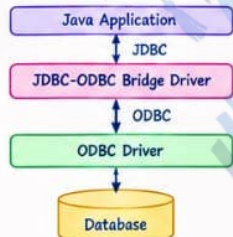
JDBC provides different types of drivers to connect with databases. Each type uses a different approach to communicate with the database.

1 Type 1 - JDBC-ODBC Bridge Driver

- Translates JDBC calls into ODBC calls.
- Requires ODBC driver and native libraries.
- Slower performance.
- Not recommended for modern applications.

Example:

`sun.jdbc.odbc.JdbcOdbcDriver`

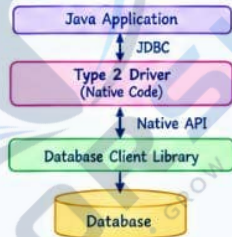


2 Type 2 - Native API Driver (Partially Java)

- Uses native code (DLL/so) to communicate with the database.
- Converts JDBC calls to native database calls.
- Better performance than Type 1.
- Platform dependent (requires native libraries).

Example:

`com.mysql.jdbc.Driver`
(older versions)

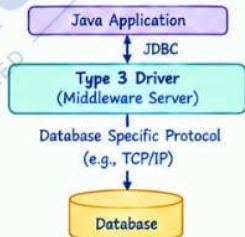


3 Type 3 - Network Protocol Driver (Middleware / Pure Java)

- Uses a middleware server (driver) that communicates with the database.
- No native libraries required.
- Platform independent.
- Better scalability and flexibility.

Example:

`com.sun.rowset.JdbcRowSetImpl`
(or vendor specific)

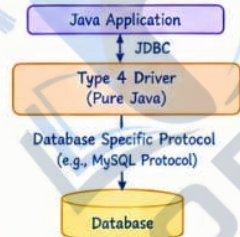


4 Type 4 - Thin Driver (Pure Java)

- Converts JDBC calls directly into database-specific protocol.
- No native code or middleware required.
- Fast and efficient.
- Platform independent.

Example:

`com.mysql.cj.jdbc.Driver`
(MySQL)

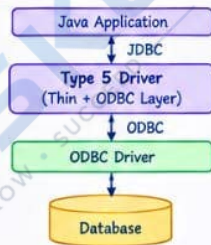


5 Type 5 - JDBC/ODBC Driver (Hybrid)

- Combines features of Type 1 and Type 4.
- Uses a thin driver (Type 4) and an ODBC layer.
- Improves portability and performance.

Example:

`com.informix.jdbc.IfxDriver`
(Informix)



Quick Comparison

Type	Name	Key Feature	Uses Native Code?	Platform Independent?	Performance	Example
1	JDBC-ODBC Bridge	Uses ODBC	Yes (ODBC)	No	Low	<code>sun.jdbc.odbc.JdbcOdbcDriver</code>
2	Native API Driver	Uses native libraries	Yes	No	Medium	<code>com.mysql.jdbc.Driver</code>
3	Network Protocol Driver	Middleware server	No	Yes	Medium - High	<code>com.sun.rowset.JdbcRowSetImpl</code>
4	Thin Driver	Direct DB communication	No	Yes	High	<code>com.mysql.cj.jdbc.Driver</code>
5	JDBC/ODBC Driver	Hybrid (Type 4 + ODBC)	Yes (ODBC)	No	Medium - High	<code>com.informix.jdbc.IfxDriver</code>



Key Takeaways

- Type 4 (Thin Driver) is most widely used in modern applications.
- Type 3 is useful for enterprise-level distributed systems.
- Type 1 and Type 2 are older and less preferred now.
- Always choose the driver based on your database, performance needs and deployment environment.

Steps to Create JDBC Application

A JDBC application connects to a database, executes SQL queries and processes the results.

1 Import JDBC Package

- Import the required classes from java.sql package.

```
import java.sql.*;
```

2 Load and Register Driver

- Load the JDBC driver class.
(Required for older versions, optional for newer versions.)

```
Class.forName("com.mysql.cj.jdbc.Driver");  
// For MySQL
```

(or use DriverManager in newer versions)

3 Establish Connection

- Use DriverManager.getConnection() with URL, username and password.

```
Connection con = DriverManager.getConnection(  
    "jdbc:mysql://localhost:3306/testdb",  
    "root", "password");
```

4 Create a Statement

- Create a Statement, PreparedStatement or CallableStatement to execute queries.

```
Statement stmt = con.createStatement();  
// or  
PreparedStatement pstmt = con.prepareStatement(  
    "SELECT * FROM student");
```

5 Execute SQL Query

- Use executeQuery() for SELECT.
- Use executeUpdate() for INSERT, UPDATE, DELETE.

```
ResultSet rs = stmt.executeQuery(  
    "SELECT * FROM student");  
// or  
int rows = stmt.executeUpdate(  
    "INSERT INTO student(name) VALUES('Rahul')");
```

6 Process the ResultSet

- Iterate through the ResultSet and retrieve data using getter methods.

```
while (rs.next()) {  
    int id = rs.getInt("id");  
    String name = rs.getString("name");  
    System.out.println(id + " " + name);  
}
```

7 Close Resources

- Close ResultSet, Statement and Connection in a finally block or using try-with-resources.

```
rs.close();  
stmt.close();  
con.close();
```

8 Handle Exceptions

- Use try-catch block to handle SQLException and other exceptions.

```
try {  
    // JDBC code  
} catch (SQLException e) {  
    e.printStackTrace();  
}
```

Complete Flow of a JDBC Application



Key Points

- JDBC provides a standard API for database access.
- Use the appropriate driver for your database.
- Always close resources to avoid memory leaks.
- For modern applications, use try-with-resources.
- Handle SQLException and other exceptions properly.

Java SQL Packages

Java provides two important packages for working with databases: `java.sql` (core JDBC package) and `javax.sql` (extension package). These packages contain the classes and interfaces needed to connect, execute queries and handle database operations.

Key Components

`java.sql` (Core Package)

- Provides the main JDBC classes and interfaces for database connectivity and basic operations.
- Important classes/interfaces: `Connection`, `Statement`, `PreparedStatement`, `CallableStatement`, `ResultSet`, `DriverManager`, `SQLException`, `ResultSetMetaData`, `DatabaseMetaData`.

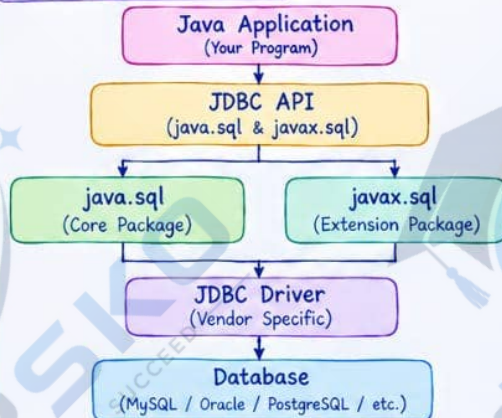
`javax.sql` (Extension Package)

- Provides additional features for enterprise and advanced database access.
- Important classes/interfaces: `DataSource`, `ConnectionPoolDataSource`, `PooledConnection`, `XADataSource`, `RowSet`.

Advantages

- Provides a standard, consistent interface for database access.
- Supports both simple and advanced database operations.
- `javax.sql` gives extra features like connection pooling, `DataSource` and `RowSet`.
- Helps in building flexible, scalable and enterprise-level applications.

Hierarchy / Relationship



How It Works (Flow)

- 1 The Java application uses classes from `java.sql` or `javax.sql` to connect to a database.
- 2 `DriverManager` loads the appropriate JDBC driver based on the URL.
- 3 A `Connection` is established with the database using `DriverManager` or `DataSource`.
- 4 `Statements` / `PreparedStatement`s are created and executed.
- 5 `ResultSet` returns the query results and `ResultSetMetaData` provides information about the result set.
- 6 Any errors are handled using `SQLException`.

Example (Java Code)

```
import java.sql.*;

public class Example {
    public static void main(String[] args) {
        try {
            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/testdb", "root", "password");
            PreparedStatement ps = con.prepareStatement(
                "SELECT id, name FROM student");
            ResultSet rs = ps.executeQuery();
            while (rs.next()) {
                System.out.println(rs.getInt("id") + " - " + rs.getString("name"));
            }
            rs.close(); ps.close(); con.close();
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Important Packages / Classes

`java.sql` (Core Package)

- `Connection` – represents a database connection.
- `Statement` – executes static SQL queries.
- `PreparedStatement` – executes parameterized queries.
- `CallableStatement` – calls stored procedures.
- `ResultSet` – holds query results.
- `DriverManager` – loads and manages JDBC drivers.
- `SQLException` – handles database related errors.
- `ResultSetMetaData` – provides metadata about result set.
- `DatabaseMetaData` – provides metadata about the database.

`javax.sql` (Extension Package)

- `DataSource` – provides a connection to the database.
- `ConnectionPoolDataSource` – supports connection pooling.
- `PooledConnection` – represents a pooled connection.
- `XADataSource` – supports distributed (XA) transactions.
- `RowSet` – provides a disconnected/scrollable result set.

Inserting & Updating Records

In JDBC, we use SQL **INSERT** and **UPDATE** statements to add new records or modify existing records in a database table. These operations are executed using the Statement or PreparedStatement object.

Important Points

- INSERT** adds a new row (record) to a table.
- UPDATE** modifies existing data in a table.
- Use **PreparedStatement** for safety, better performance and to avoid SQL injection.
- After executing INSERT or UPDATE, the method **executeUpdate()** returns the number of rows affected.
- If no record is affected, it returns 0.

Inserting Records (INSERT)

SQL Syntax

```
INSERT INTO table_name (column1, column2, ...)  
VALUES (value1, value2, ...);
```

Example

```
String sql = "INSERT INTO student  
(name, age, course) VALUES ('Rohit', 20, 'BSc')";
```

Table before Inserting

id	name	age	course
1	Aditi	19	BSc



Table after Inserting

id	name	age	course
1	Aditi	19	BSc
2	Rohit	20	BSc

Updating Records (UPDATE)

SQL Syntax

```
UPDATE table_name  
SET column1 = value1, column2 = value2, ...  
WHERE condition;
```

Example

```
String sql = "UPDATE student  
SET course = 'MCA' WHERE id = 1";
```

Table before Update

id	name	age	course
1	Aditi	19	BSc



Table after Update

id	name	age	course
1	Aditi	19	MCA

Steps (Common for Both)

- Load the JDBC driver (Class.forName()).
- Get the connection using DriverManager.getConnection().
- Create a Statement or PreparedStatement object.
- Write the SQL query (INSERT or UPDATE).
- Execute the query using executeUpdate().
- Close the resources (ResultSet, Statement, Connection).

Complete Example (Insert & Update)

```
import java.sql.*;  
public class InsertUpdateExample {  
    public static void main(String[] args) {  
        try {  
            Class.forName("com.mysql.cj.jdbc.Driver");  
            Connection con = DriverManager.getConnection(  
                "jdbc:mysql://localhost:3306/student", "root", "password");  
            Statement stmt = con.createStatement();  
            // Insert record  
            String insertSql = "INSERT INTO student(name, age, course) "+  
                "VALUES('Rahul', 21, 'BCA')";  
            int rows1 = stmt.executeUpdate(insertSql);  
            System.out.println("Inserted Rows: " + rows1);  
            // Update record  
            String updateSql = "UPDATE student SET course='MCA' WHERE id=2";  
            int rows2 = stmt.executeUpdate(updateSql);  
            System.out.println("Updated Rows: " + rows2);  
            stmt.close(); con.close();  
        } catch (Exception e) {  
            e.printStackTrace();  
        }  
    }  
}
```

Key Takeaways

- INSERT** → adds new record.
- UPDATE** → modifies existing record.
- Use **PreparedStatement** for security and performance.
- executeUpdate()** returns number of affected rows.
- Always close **Connection**, **Statement** and **ResultSet**.

Selecting & Modifying Records

In JDBC, we use SELECT to retrieve data from a table and UPDATE / DELETE to modify or remove existing records.

These operations are executed using the Statement or PreparedStatement object.

Key Components

1 SELECT

- Used to fetch data from one or more tables.

2 UPDATE

- Used to modify existing data in a table.

3 DELETE

- Used to remove records from a table.

4 WHERE Clause

- Specifies the condition for selecting, updating or deleting records.

5 ResultSet

- Holds the data returned by a SELECT query.

Selecting Records (SELECT)

SQL Syntax

```
SELECT column1, column2, ...  
FROM table_name  
WHERE condition;
```

Example (Select All Records)

```
String sql = "SELECT * FROM students";  
ResultSet rs = stmt.executeQuery(sql);
```

Example Output

id	name	age	course
1	Rohit	20	BSc
2	Aditya	21	BCA
3	Sneha	19	BSc

Example (With Condition)

```
String sql = "SELECT * FROM students  
WHERE course = 'BSc'";  
ResultSet rs = stmt.executeQuery(sql);
```

Output (Only BSc Students)

id	name	age	course
1	Rohit	20	BSc
3	Sneha	19	BSc

Modifying Records (UPDATE)

SQL Syntax

```
UPDATE table_name  
SET column1 = value1, column2 = value2, ...  
WHERE condition;
```

Example

```
String sql = "UPDATE students  
SET course = 'MCA' WHERE id = 2";  
int rows = stmt.executeUpdate(sql);
```

💡 executeUpdate() returns the number of rows affected.

Before Update

id	name	age	course
2	Aditya	21	BCA

After Update

id	name	age	course
2	Aditya	21	MCA

Deleting Records (DELETE)

SQL Syntax

```
DELETE FROM table_name  
WHERE condition;
```

Example

```
String sql = "DELETE FROM students  
WHERE id = 3";  
int rows = stmt.executeUpdate(sql);
```



Always use **WHERE** clause. Without it, all records will be deleted.

Complete Example (Select, Update & Delete)

```
import java.sql.*;  
public class ModifyRecordsExample {  
    public static void main(String[] args) {  
        try {  
            Connection con = DriverManager.getConnection(  
                "jdbc:mysql://localhost:3306/student", "root", "password");  
            Statement stmt = con.createStatement();  
  
            // 1. Select  
            ResultSet rs = stmt.executeQuery("SELECT * FROM students");  
            while (rs.next()) {  
                System.out.println(rs.getInt("id") + " * " + rs.getString("name") + " * " + rs.getString("course"));  
            }  
  
            // 2. Update  
            int update = stmt.executeUpdate("UPDATE students SET course = 'MCA' WHERE id = 2");  
            System.out.println("Rows updated: " + update);  
  
            // 3. Delete  
            int delete = stmt.executeUpdate("DELETE FROM students WHERE id = 3");  
            System.out.println("Rows deleted: " + delete);  
            rs.close(); stmt.close(); con.close();  
        } catch (Exception e) {  
            e.printStackTrace();  
        }  
    }  
}
```

Important Points

- ✔ Use SELECT to fetch data, UPDATE to modify and DELETE to remove data.
- ✔ Always use WHERE clause for UPDATE and DELETE.
- ✔ executeUpdate() returns the number of affected rows.
- ✔ Use PreparedStatement for safety and better performance.
- ✔ Handle SQLExceptions properly.

★ Key Takeaway

SELECT → Read data
UPDATE / DELETE → Modify or remove data
(Always use WHERE clause!)